



C#.NET

Mr. Bangar Raju

 **NARESHO**
technologies

Access Specifiers

Understanding Access Specifiers in C#.Net: A Comprehensive Guide

By Albert Harmon

Published September 10, 2016

🕒 4 min read

[Education](#)



TLDR This blog post provides an in-depth exploration of access specifiers in C#.Net, detailing their definitions, scopes, and practical examples through various classes and methods. It covers five access specifiers: private, internal, protected,

protected internal, and public, explaining their accessibility and usage in different scenarios.

Access specifiers are essential components in C#.Net that define the accessibility of classes and their members. In this guide, we will explore what access specifiers are, the different types available in C#, and how they can be practically applied in programming.

What are Access Specifiers?

Access specifiers are special modifiers that determine the scope of a type and its members. They control the visibility and accessibility of classes, methods, and other members within a program. In C#, there are five primary access specifiers:

1. **Private**
2. **Internal**
3. **Protected**
4. **Protected Internal**
5. **Public**

Each of these specifiers has a unique scope that dictates where and how they can be accessed.

The Five Access Specifiers

1. Private

- **Scope:** Members declared as private are accessible only within the class in which they are defined.
- **Default Scope:** The default scope for any member of a class is private.
- **Important Note:** You cannot declare a class as private.

2. Internal

- **Scope:** Members declared as internal are accessible only within the same assembly (project). This means that both child and non-child classes can access internal members, but only if they are within the same project.

3. Protected

- **Scope:** Members declared as protected are accessible within the class and by derived classes (child classes). Non-child classes cannot access protected members.
- **Important Note:** You cannot declare a class as protected.

4. Protected Internal

- **Scope:** Members declared as protected internal can be accessed by both derived classes and any class within the same assembly.

This means that if either protected or internal access is granted, the member can be accessed.

5. Public

- **Scope:** Members declared as public are accessible from anywhere in the program. There are no restrictions on access.

Practical Examples

To illustrate the use of access specifiers, let's create a console application in Visual Studio with various classes demonstrating each specifier.

Creating the Classes

1. **Class Program:** This class will contain methods with different access specifiers.

```
public class Program
{
    private void PrivateMethod() { }
    internal void InternalMethod() { }
    protected void ProtectedMethod() { }
    protected internal void ProtectedInternalMethod() { }
```

```
        public void PublicMethod() { }  
    }  
}
```

2. **Class Two:** This class will inherit from Program and demonstrate access to its members.

```
public class ClassTwo : Program  
{  
    public void TestAccess()  
    {  
        // Can access:  
        InternalMethod();  
        ProtectedMethod();  
        ProtectedInternalMethod();  
        PublicMethod();  
        // Cannot access PrivateMethod()  
    }  
}
```

3. **Class Three:** This class will attempt to access Program's members without inheritance.

```
public class ClassThree  
{  
    public void TestAccess()
```

```

    {
        Program p = new Program();
        // Can access:
        p.PublicMethod();
        // Cannot access InternalMethod(), ProtectedMethod
    }
}

```

4. **Class Four:** This class will demonstrate access from a different project.

```

public class ClassFour : Program
{
    public void TestAccess()
    {
        // Can access:
        ProtectedMethod();
        ProtectedInternalMethod();
        PublicMethod();
        // Cannot access PrivateMethod() or InternalMethod
    }
}

```



5. **Class Five:** This class will attempt to access Program's members from a different project without inheritance.

```
public class ClassFive
{
    public void TestAccess()
    {
        Program p = new Program();
        // Can only access:
        p.PublicMethod();
        // Cannot access PrivateMethod(), InternalMethod()
    }
}
```



Summary of Access Scopes

- **Private:** Accessible only within the class.
- **Internal:** Accessible within the same project.
- **Protected:** Accessible within the class and derived classes.
- **Protected Internal:** Accessible within the project or derived classes.
- **Public:** Accessible from anywhere.

Conclusion

Understanding access specifiers is crucial for effective programming in C#.Net. They help in encapsulating data and controlling access to class

members, which is fundamental for maintaining the integrity and security of your code. By applying the correct access specifiers, developers can ensure that their classes are used as intended, preventing unauthorized access and modifications.

This guide has provided a comprehensive overview of access specifiers in C#.Net, along with practical examples to illustrate their usage. By mastering these concepts, you can enhance your programming skills and write more robust and secure applications.